

Exercício sobre Testes de Unidade

Testes e Estratégias de Teste

Engenharia de Software II

O objetivo é utilizar o JUnit para realizar testes de unidade em Java e compreender a diferença prática entre testes com e sem Mocks utilizando Mockito.

Além disso, o JaCoCo é utilizado para mensurar a cobertura de código seguindo os conceitos do livro Engenharia de Software Moderna, do Prof. Marco Tulio Valente.

Passo 1: Preparação do Ambiente (Instalação do Maven)

O Maven é a ferramenta que automatizará o gerenciamento de bibliotecas (JUnit, Mockito) e o build do projeto.

1. Verifique se o Maven está instalado:

Shell

```
mvn -version
```

Certifique-se de ter a seguinte extensão instalada no seu VS Code:

- Extension Pack for Java (Microsoft)
- Test Runner for Java (Microsoft)

Passo 2: Criando o Projeto Maven no VS Code

Para evitar problemas de caminhos e arquivos, deixe o VS Code criar a estrutura padrão do Maven.

1. No VS Code, abra a Paleta de Comandos: Ctrl + Shift + P.
2. Digite e selecione: **Java: Create Java Project...**
3. Escolha a opção: **Maven**
4. Escolha o modelo (*archetype*): **maven-archetype-quickstart** (pode dar Enter na versão mais recente).
5. Preencha os campos conforme solicitado (aperte Enter após digitar cada um):
 - **Group Id:** engsoft
 - **Artifact Id:** sistema-vendas
6. O VS Code pedirá para escolher uma pasta no seu computador. Escolha o local desejado e clique em **Select Destination Folder**.
7. No canto inferior direito, aparecerá uma notificação. Clique em **Open** para abrir o projeto que acabou de ser gerado.

8.

Passo 3: Configurando as Dependências (pom.xml)

Abra o arquivo pom.xml localizado na raiz do projeto, apague todo o seu conteúdo e substitua pelo código em anexo. Este arquivo já inclui o **JUnit 4**, o **Mockito** e o plugin do **JaCoCo**:

Salve o arquivo. Se o VS Code perguntar se deseja atualizar o projeto Maven, clique em **Yes/Update**.

Passo 4: Criando os Arquivos nas Pastas Corretas

Parte A: Código do Sistema

Crie os dois códigos do sistema a seguir na pasta **src/main/java/engsoft**:

GatewayPagamento.java

```
Java
package engsoft;

public interface GatewayPagamento {
    boolean cobrar(String cartao, double valor);
}
```

CartaoPagamento.java

```
Java
package engsoft;

public class CartaoPagamento implements GatewayPagamento{

    public CartaoPagamento() {
    }

    @Override
    public boolean cobrar(String cartao, double valor) {
        return true;
    }
}
```

ProcessadorVendas.java

Java

```
package engsoft;

public class ProcessadorVendas{

    private GatewayPagamento gateway;

    public ProcessadorVendas(GatewayPagamento gateway) {
        this.gateway = gateway;
    }

    public String finalizarVenda(double valorTotal, String numeroCartao) {
        if (valorTotal <= 0) {
            return "VENDA_REJEITADA: Valor inválido";
        }

        boolean sucesso = gateway.cobrar(numeroCartao, valorTotal);

        if (sucesso) {
            return "VENDA_SUCESSO";
        } else {
            return "VENDA_REJEITADA: Cartão sem saldo ou inválido";
        }
    }
}
```

Parte B: Código de Teste

Crie os dois arquivos de teste na pasta **src/test/java/engsoft**:

Abordagem manual/tradicional:

ProcessadorVendasSemMockTest.java

Java

```
package engsoft;

import org.junit.Test;
import static org.junit.Assert.*;

public class ProcessadorVendasSemMockTest {

    @Test
    public void testFinalizarVendaComSucesso() {
        GatewayPagamento gatewayReal = new CartaoPagamento();

        ProcessadorVendas processador = new ProcessadorVendas(gatewayReal);
        String resultado = processador.finalizarVenda(100.0, "1234-5678-9012-3456");
    }
}
```

```
        assertEquals("VENDA_SUCESSO", resultado);
    }
}
```

Abordagem moderna com **Mockito**

ProcessadorVendasComMockTest.java

```
Java
package engsoft;

import org.junit.Before;
import org.junit.Test;
import static org.junit.Assert.*;
import static org.mockito.Mockito.*;

public class ProcessadorVendasComMockTest {

    private GatewayPagamento gatewayMock;
    private ProcessadorVendas processador;

    @Before
    public void init() {
        // Criação do objeto Mock
        gatewayMock = mock(GatewayPagamento.class);
        processador = new ProcessadorVendas(gatewayMock);
    }

    @Test
    public void testFinalizarVendaComSucesso() {

        when(gatewayMock.cobrar(anyString(), anyDouble()))
            .thenReturn(true);

        String resultado =
            processador.finalizarVenda(100.0, "1234");

        assertEquals("VENDA_SUCESSO", resultado);
    }
}
```

Passo 5: Executando os Testes e Verificando a Cobertura

Agora rode o ciclo completo do Maven pelo terminal. Garanta que você está na pasta raiz (sistema-vendas) e execute o comando:

mvn clean test

O terminal deverá exibir o andamento dos testes e finalizar com a mensagem: **BUILD SUCCESS**.

Abra o arquivo *index.html* na pasta *sistema-vendas/target/site/jacoco*.

Passo 6: Crie um Teste que Falhe

Com esse exemplo funcionando, escreva um método de teste na classe *ProcessadorVendasComMockTest* que **falhe**.

No seu terminal, você deve ver algo como:

```
Tests run: 2, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 1.402 sec <<< FAILURE!
testFinalizarVendaSemSucesso(engsoft.ProcessadorVendasComMockTest) Time elapsed: 0.009
sec <<< FAILURE!
org.junit.ComparisonFailure: expected: ----- but was:-----
```

RESULTADOS

Envie em um PDF:

- Análise do uso de Mocks para os testes unitários;
- A classe *ProcessadorVendasComMockTest*, incluindo o **método de teste** que você criou;
- A mensagem de saída do seu terminal durante o teste que passou e o teste que falhou.

Adicionalmente, envie o arquivo **index.html** mencionado no **passo 5**.